==========:

LOANTAP - LOGISTIC REGRESSION CASE STUDY

==========:

Import Libraries

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve,
                             precision_recall_curve, average_precision_score)

sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 5)
print("✅ Libraries loaded!")
```

```
✅ Libraries loaded!
```

Load Dataset

In [3]:
```
df = pd.read_csv('logistic_regression.csv')
print("Shape:", df.shape)
df.head()
```

```
Shape: (396030, 27)
```

Out[3]:

	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	emp_lengt
0	10000.0	36 months	11.44	329.48	B	B4	Marketing	10+ yea
1	8000.0	36 months	11.99	265.68	B	B5	Credit analyst	4 yea
2	15600.0	36 months	10.49	506.97	B	B3	Statistician	< 1 ye
3	7200.0	36 months	6.49	220.65	A	A2	Client Advocate	6 yea
4	24375.0	60 months	17.27	609.33	C	C5	Destiny Management Inc.	9 yea

5 rows × 27 columns

Basic Structure

In [ ]:
```
print("=== Data Types ===")
print(df.dtypes)
```

```
=== Data Types ===
loan_amnt               float64
term                     object
int_rate                float64
installment             float64
grade                    object
sub_grade                object
emp_title                object
emp_length               object
home_ownership           object
annual_inc              float64
verification_status      object
issue_d                  object
loan_status              object
purpose                  object
title                    object
dti                     float64
earliest_cr_line         object
open_acc                float64
pub_rec                 float64
revol_bal               float64
revol_util              float64
total_acc               float64
initial_list_status      object
application_type         object
mort_acc                float64
pub_rec_bankruptcies    float64
address                  object
dtype: object
```

In [6]:
```
print("\n=== Missing Values ===")
print(df.isnull().sum())
```

```
=== Missing Values ===
loan_amnt                   0
term                        0
int_rate                    0
installment                 0
grade                       0
sub_grade                   0
emp_title               22927
emp_length              18301
home_ownership              0
annual_inc                  0
verification_status         0
issue_d                     0
loan_status                 0
purpose                     0
title                    1756
dti                         0
earliest_cr_line            0
open_acc                    0
pub_rec                     0
revol_bal                   0
revol_util                276
total_acc                   0
initial_list_status         0
application_type            0
mort_acc                37795
pub_rec_bankruptcies      535
address                     0
dtype: int64
```

In [7]:
```
print("\n=== Duplicate Rows ===")
print(df.duplicated().sum())
```

```
=== Duplicate Rows ===
0
```

In [8]:
```
print("\n=== Statistical Summary ===")
df.describe().T
```

```
=== Statistical Summary ===
```

Out[8]:

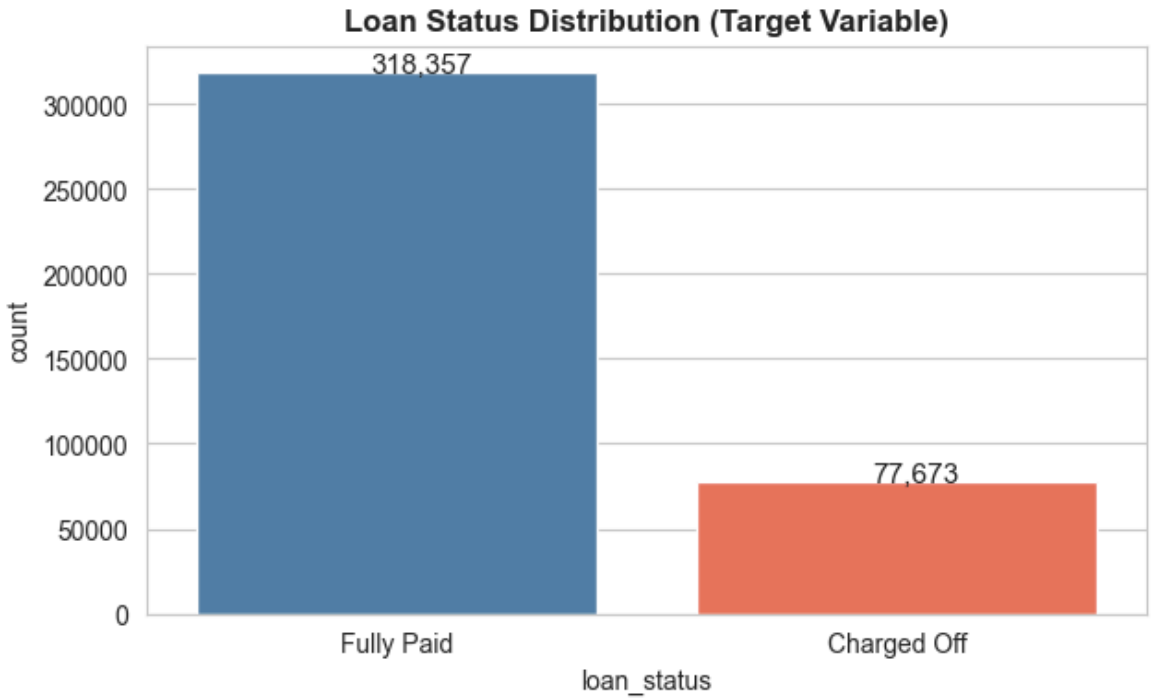
	count	mean	std	min	25%	50%
loan_amnt	396030.0	14113.888089	8357.441341	500.00	8000.00	12000.00
int_rate	396030.0	13.639400	4.472157	5.32	10.49	13.33
installment	396030.0	431.849698	250.727790	16.08	250.33	375.43
annual_inc	396030.0	74203.175798	61637.621158	0.00	45000.00	64000.00
dti	396030.0	17.379514	18.019092	0.00	11.28	16.91
open_acc	396030.0	11.311153	5.137649	0.00	8.00	10.00
pub_rec	396030.0	0.178191	0.530671	0.00	0.00	0.00
revol_bal	396030.0	15844.539853	20591.836109	0.00	6025.00	11181.00
revol_util	395754.0	53.791749	24.452193	0.00	35.80	54.80
total_acc	396030.0	25.414744	11.886991	2.00	17.00	24.00
mort_acc	358235.0	1.813991	2.147930	0.00	0.00	1.00
pub_rec_bankruptcies	395495.0	0.121648	0.356174	0.00	0.00	0.00

Target Variable Distribution

In [10]:
```
print("=== Target: loan_status ===")
print(df['loan_status'].value_counts())
print("\nFully Paid %:", round(df['loan_status'].value_counts(normalize=True)['F
```

```
=== Target: loan_status ===
loan_status
Fully Paid     318357
Charged Off     77673
Name: count, dtype: int64

Fully Paid %: 80.39
```

In [11]:
```
plt.figure(figsize=(7,4))
ax = sns.countplot(x='loan_status', data=df, palette=['steelblue','tomato'])
for p in ax.patches:
    ax.annotate(f'{p.get_height():,.0f}', (p.get_x()+0.35, p.get_height()+500),
plt.title('Loan Status Distribution (Target Variable)', fontweight='bold')
plt.savefig('01_target_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
print("💡 ~80.4% loans are Fully Paid; ~19.6% are Charged Off — class imbalance
```

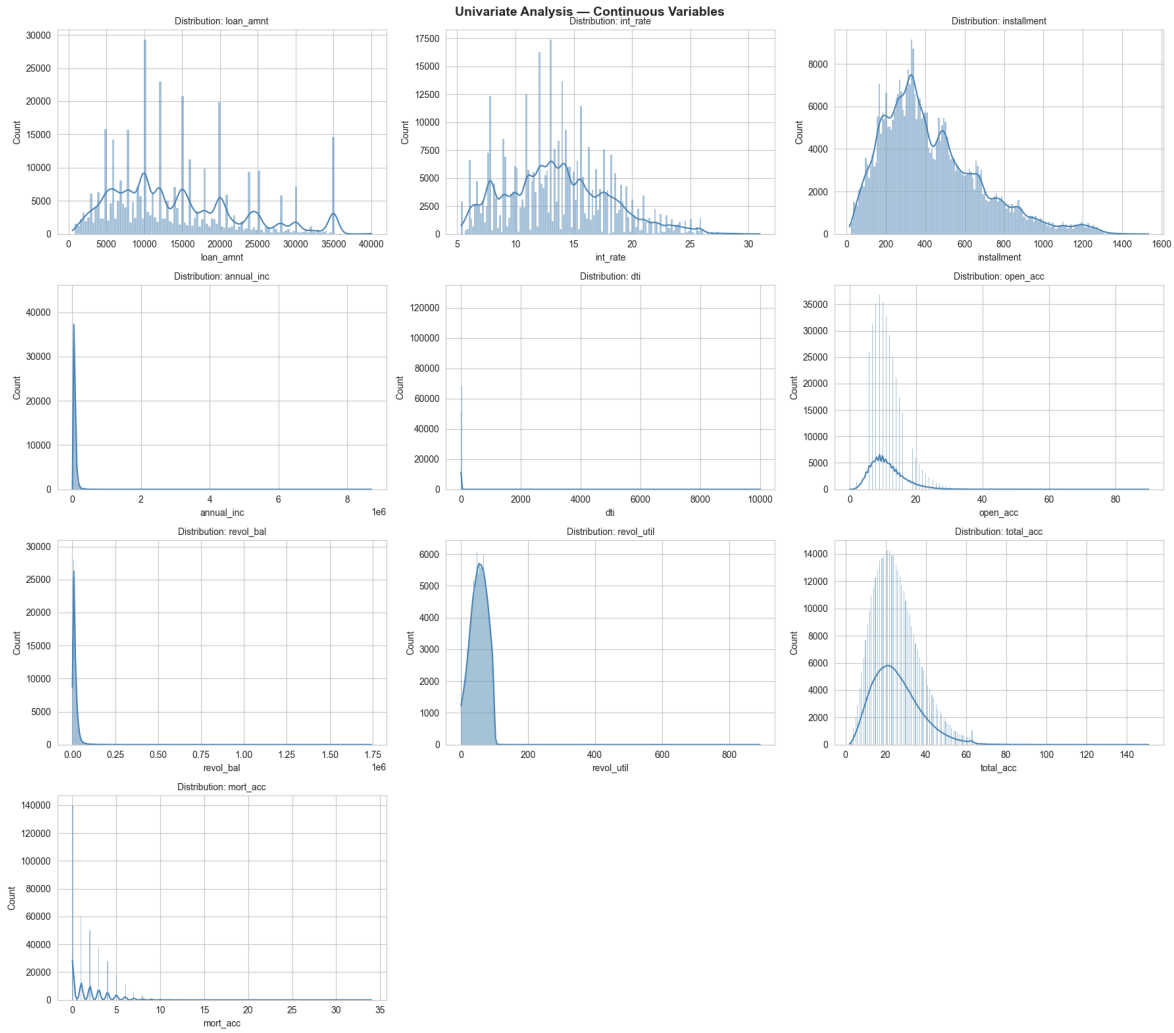


💡 ~80.4% loans are Fully Paid; ~19.6% are Charged Off — class imbalance presen
t.

EXPLORATORY DATA ANALYSIS

Univariate – Continuous Variables

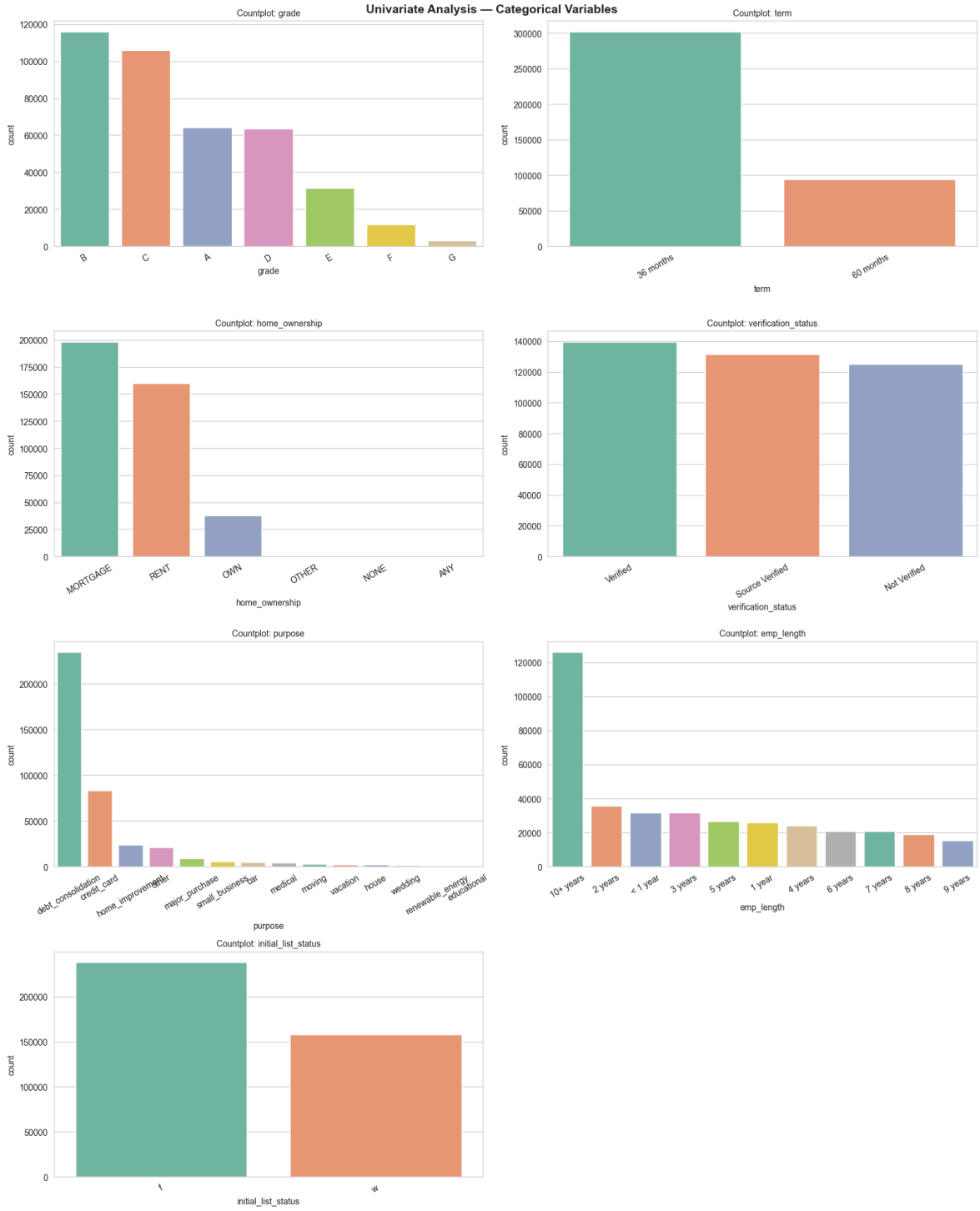
In [12]:
```
num_cols = ['loan_amnt', 'int_rate', 'installment', 'annual_inc', 'dti',
            'open_acc', 'revol_bal', 'revol_util', 'total_acc', 'mort_acc']

fig, axes = plt.subplots(4, 3, figsize=(18, 16))
axes = axes.flatten()
for i, col in enumerate(num_cols):
    sns.histplot(df[col].dropna(), kde=True, ax=axes[i], color='steelblue')
    axes[i].set_title(f'Distribution: {col}', fontsize=10)
for j in range(len(num_cols), len(axes)):
    axes[j].set_visible(False)
plt.suptitle('Univariate Analysis — Continuous Variables', fontsize=14, fontweig
plt.tight_layout()
plt.savefig('02_univariate_continuous.png', dpi=150, bbox_inches='tight')
plt.show()
print("💡 loan_amnt, installment, annual_inc are right-skewed. int_rate is rough
```



💡 loan_amnt, installment, annual_inc are right-skewed. int_rate is roughly normal.

Univariate – Categorical Variables

In [13]:
```
cat_cols = ['grade', 'term', 'home_ownership', 'verification_status',
            'purpose', 'emp_length', 'initial_list_status']

fig, axes = plt.subplots(4, 2, figsize=(16, 20))
axes = axes.flatten()
for i, col in enumerate(cat_cols):
    order = df[col].value_counts().index
    sns.countplot(x=col, data=df, ax=axes[i], palette='Set2', order=order)
    axes[i].set_title(f'Countplot: {col}', fontsize=10)
    axes[i].tick_params(axis='x', rotation=30)
for j in range(len(cat_cols), len(axes)):
    axes[j].set_visible(False)
plt.suptitle('Univariate Analysis — Categorical Variables', fontsize=14, fontwei
plt.tight_layout()
plt.savefig('03_univariate_categorical.png', dpi=150, bbox_inches='tight')
plt.show()
print("💡 Most loans are 36-month term, Grade B/C, MORTGAGE home ownership.")
print("💡 Majority purpose is debt consolidation.")
```

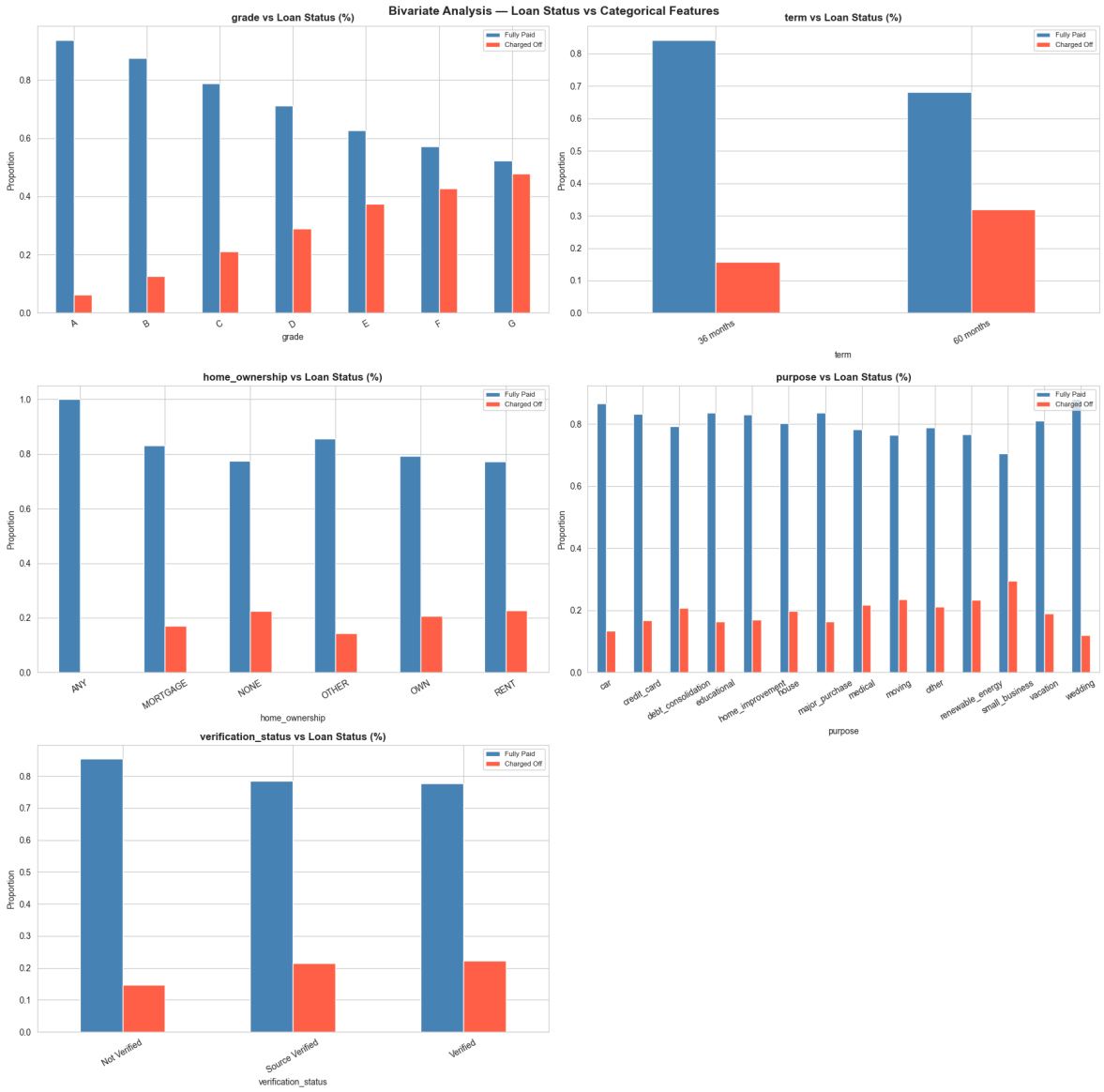


💡 Most loans are 36-month term, Grade B/C, MORTGAGE home ownership.
💡 Majority purpose is debt consolidation.

Bivariate – Loan Status vs Categorical

In [14]:
```
biv_cats = ['grade', 'term', 'home_ownership', 'purpose', 'verification_status']

fig, axes = plt.subplots(3, 2, figsize=(18, 18))
axes = axes.flatten()
for i, col in enumerate(biv_cats):
    ct = df.groupby(col)['loan_status'].value_counts(normalize=True).unstack()
    ct[['Fully Paid', 'Charged Off']].plot(kind='bar', ax=axes[i],
        color=['steelblue','tomato'], edgecolor='white')
    axes[i].set_title(f'{col} vs Loan Status (%)', fontweight='bold')
    axes[i].set_ylabel('Proportion')
```

```
    axes[i].tick_params(axis='x', rotation=30)
    axes[i].legend(loc='upper right', fontsize=8)
axes[5].set_visible(False)
plt.suptitle('Bivariate Analysis — Loan Status vs Categorical Features',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('04_bivariate_categorical.png', dpi=150, bbox_inches='tight')
plt.show()
print("💡 Grade A has highest Fully Paid rate (~94%). Grade G has highest Charge
print("💡 60-month loans have higher default rate than 36-month loans.")
```

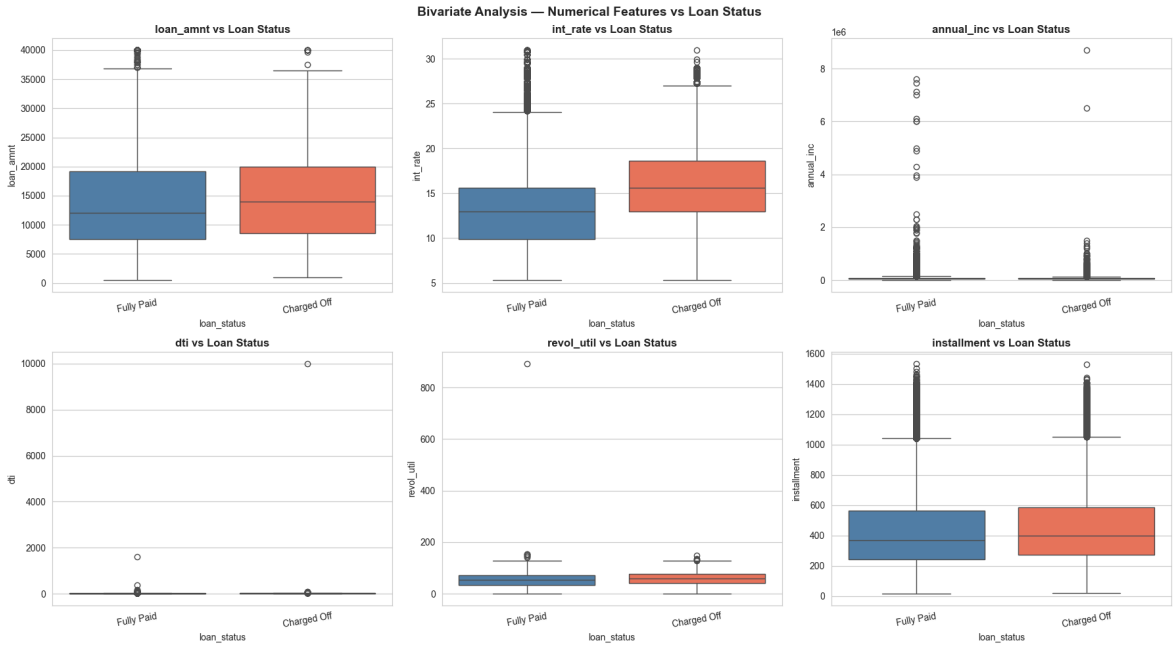



```
💡 Grade A has highest Fully Paid rate (~94%). Grade G has highest Charged Off r
ate (~43%).
💡 60-month loans have higher default rate than 36-month loans.
```

Bivariate – Loan Status vs Numerical

In [15]:
```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()
num_biv = ['loan_amnt', 'int_rate', 'annual_inc', 'dti', 'revol_util', 'installm
for i, col in enumerate(num_biv):
    sns.boxplot(x='loan_status', y=col, data=df, ax=axes[i],
                palette=['steelblue','tomato'])
    axes[i].set_title(f'{col} vs Loan Status', fontweight='bold')
```

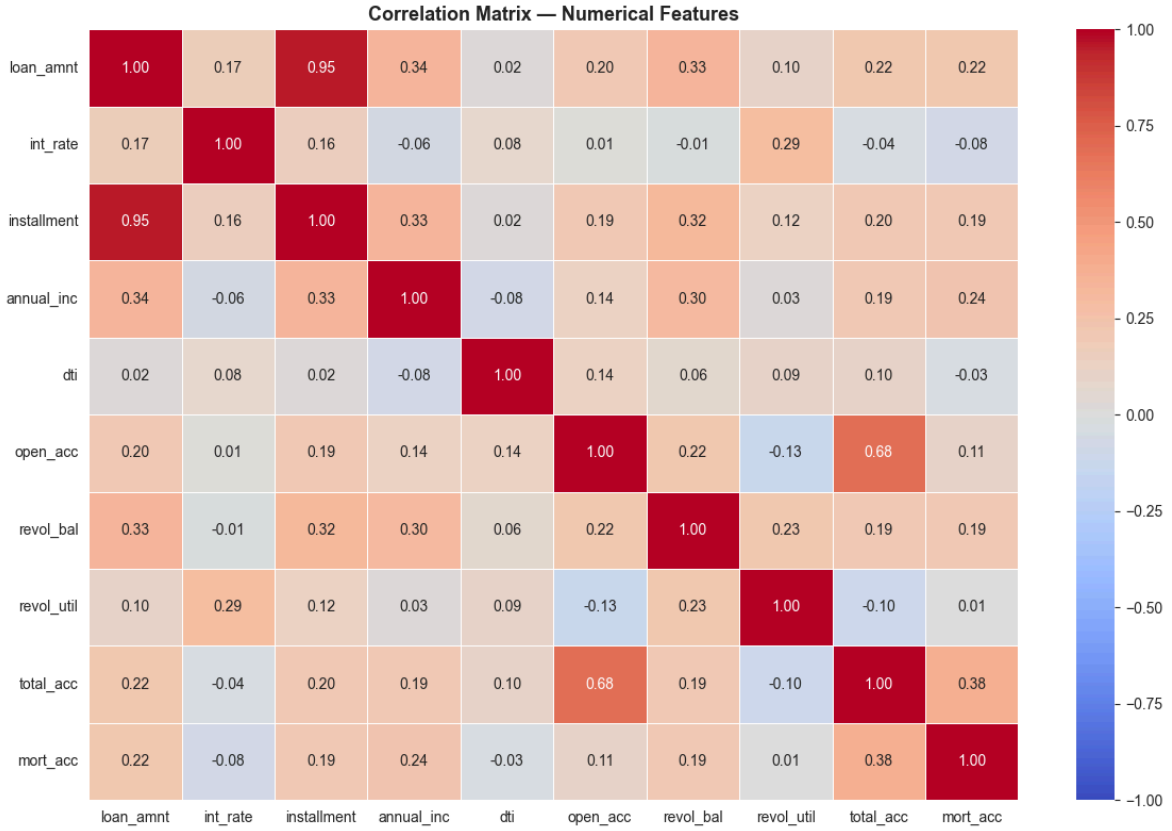
```
    axes[i].tick_params(axis='x', rotation=10)
plt.suptitle('Bivariate Analysis — Numerical Features vs Loan Status',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('05_bivariate_numerical.png', dpi=150, bbox_inches='tight')
plt.show()
print("💡 Charged Off loans have higher int_rate, dti, revol_util.")
print("💡 Fully Paid loans have slightly higher annual_inc.")
```




```
💡 Charged Off loans have higher int_rate, dti, revol_util.
💡 Fully Paid loans have slightly higher annual_inc.
```

Correlation Heatmap

In [16]:
```
plt.figure(figsize=(12, 8))
corr = df[num_cols].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            linewidths=0.5, vmin=-1, vmax=1)
plt.title('Correlation Matrix — Numerical Features', fontsize=13, fontweight='bo
plt.tight_layout()
plt.savefig('06_correlation_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
print("💡 loan_amnt & installment are highly correlated (0.95) — one may be drop
print("💡 open_acc & total_acc have moderate correlation (0.68).")
```



💡 loan_amnt & installment are highly correlated (0.95) – one may be dropped.
💡 open_acc & total_acc have moderate correlation (0.68).

DATA PREPROCESSING

Drop Irrelevant / Leakage Columns

In [17]:
```
df.drop(columns=['emp_title', 'title', 'issue_d', 'earliest_cr_line',
                 'address', 'sub_grade'], inplace=True)
print("✅ Dropped: emp_title, title, issue_d, earliest_cr_line, address, sub_gra
print("Remaining columns:", df.columns.tolist())
```

```
✅ Dropped: emp_title, title, issue_d, earliest_cr_line, address, sub_grade
Remaining columns: ['loan_amnt', 'term', 'int_rate', 'installment', 'grade', 'emp
_length', 'home_ownership', 'annual_inc', 'verification_status', 'loan_status',
'purpose', 'dti', 'open_acc', 'pub_rec', 'revol_bal', 'revol_util', 'total_acc',
'initial_list_status', 'application_type', 'mort_acc', 'pub_rec_bankruptcies']
```

Feature Engineering — Flags

In [19]:
```
# Create binary flags as instructed
df['pub_rec_flag']            = (df['pub_rec'] > 0).astype(int)
df['mort_acc_flag']           = (df['mort_acc'] > 0).astype(int)
df['pub_rec_bankruptcies_flag'] = (df['pub_rec_bankruptcies'] > 0).astype(int)

print("✅ Binary flags created:")
```

```
✅ Binary flags created:
```

In [20]:
```
print("pub_rec_flag:\n", df['pub_rec_flag'].value_counts())
```

```
pub_rec_flag:
 pub_rec_flag
0    338272
1     57758
Name: count, dtype: int64
```

In [21]:
```
print("mort_acc_flag:\n", df['mort_acc_flag'].value_counts())
```

```
mort_acc_flag:
 mort_acc_flag
1    218458
0    177572
Name: count, dtype: int64
```

In [22]:
```
print("pub_rec_bankruptcies_flag:\n", df['pub_rec_bankruptcies_flag'].value_coun
```

```
pub_rec_bankruptcies_flag:
 pub_rec_bankruptcies_flag
0    350915
1     45115
Name: count, dtype: int64
```

Missing Value Treatment

In [23]:
```
print("Missing values before treatment:")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

```
Missing values before treatment:
emp_length              18301
revol_util                276
mort_acc                37795
pub_rec_bankruptcies      535
dtype: int64
emp_length              18301
revol_util                276
mort_acc                37795
pub_rec_bankruptcies      535
dtype: int64
```

In [25]:
```
# emp_length: fill with mode
df['emp_length'].fillna(df['emp_length'].mode()[0], inplace=True)
# revol_util: fill with median
df['revol_util'].fillna(df['revol_util'].median(), inplace=True)

# mort_acc: fill with median
df['mort_acc'].fillna(df['mort_acc'].median(), inplace=True)

# pub_rec_bankruptcies: fill with 0
df['pub_rec_bankruptcies'].fillna(0, inplace=True)
```

In [26]:
```
print("\nMissing values after treatment:")
print(df.isnull().sum()[df.isnull().sum() > 0])
print("✅ All missing values treated.")
```

```
Missing values after treatment:
Series([], dtype: int64)
✅ All missing values treated.
Series([], dtype: int64)
✅ All missing values treated.
```

Outlier Treatment

In [ ]:
```
print("Shape before outlier treatment:", df.shape)

for col in ['annual_inc', 'revol_bal', 'open_acc', 'total_acc']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    upper = Q3 + 3 * IQR   # using 3x IQR for large dataset
    df = df[df[col] <= upper]

print("Shape after outlier treatment:", df.shape)
```

```
Shape before outlier treatment: (396030, 24)
Shape after outlier treatment: (382594, 24)
```

Encode Target Variable

In [28]:
```
df['loan_status'] = df['loan_status'].map({'Fully Paid': 0, 'Charged Off': 1})
print("Target encoding: Fully Paid=0, Charged Off=1")
print(df['loan_status'].value_counts())
```

```
Target encoding: Fully Paid=0, Charged Off=1
loan_status
0    307070
1     75524
Name: count, dtype: int64
```

Encode Categorical Features

In [29]:
```
# Binary encode term
df['term'] = df['term'].str.strip().str.replace(' months','').astype(int)

# Ordinal encode grade
grade_map = {'A':1,'B':2,'C':3,'D':4,'E':5,'F':6,'G':7}
df['grade'] = df['grade'].map(grade_map)

# Ordinal encode emp_length
emp_map = {'< 1 year':0,'1 year':1,'2 years':2,'3 years':3,'4 years':4,
           '5 years':5,'6 years':6,'7 years':7,'8 years':8,'9 years':9,'10+ year
df['emp_length'] = df['emp_length'].map(emp_map)

# One-hot encode remaining categoricals
cat_to_encode = ['home_ownership','verification_status','purpose',
                 'initial_list_status','application_type']
df = pd.get_dummies(df, columns=cat_to_encode, drop_first=True)

print("✅ Encoding done. Shape:", df.shape)
print("Columns:", df.columns.tolist())
```

```
✅ Encoding done. Shape: (382594, 42)
Columns: ['loan_amnt', 'term', 'int_rate', 'installment', 'grade', 'emp_length',
'annual_inc', 'loan_status', 'dti', 'open_acc', 'pub_rec', 'revol_bal', 'revol_ut
il', 'total_acc', 'mort_acc', 'pub_rec_bankruptcies', 'pub_rec_flag', 'mort_acc_f
lag', 'pub_rec_bankruptcies_flag', 'home_ownership_MORTGAGE', 'home_ownership_NON
E', 'home_ownership_OTHER', 'home_ownership_OWN', 'home_ownership_RENT', 'verific
ation_status_Source Verified', 'verification_status_Verified', 'purpose_credit_ca
rd', 'purpose_debt_consolidation', 'purpose_educational', 'purpose_home_improveme
nt', 'purpose_house', 'purpose_major_purchase', 'purpose_medical', 'purpose_movin
g', 'purpose_other', 'purpose_renewable_energy', 'purpose_small_business', 'purpo
se_vacation', 'purpose_wedding', 'initial_list_status_w', 'application_type_INDIV
IDUAL', 'application_type_JOINT']
```

Train-Test Split

In [30]:
```python
X = df.drop('loan_status', axis=1)
y = df['loan_status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Train: {X_train.shape}, Test: {X_test.shape}")
print("Train target distribution:\n", y_train.value_counts(normalize=True))
```

```
Train: (306075, 41), Test: (76519, 41)
Train target distribution:
 loan_status
0    0.802601
1    0.197399
Name: proportion, dtype: float64
```

Feature Scaling

In [31]:
```python
scaler = MinMaxScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)
print("✅ MinMaxScaler applied.")
```

```
✅ MinMaxScaler applied.
```

MODEL BUILDING

Logistic Regression

In [32]:
```python
model = LogisticRegression(max_iter=1000, random_state=42, class_weight='balance
model.fit(X_train_sc, y_train)
print("✅ Logistic Regression model trained.")
print("Intercept:", model.intercept_)
```

```
✅ Logistic Regression model trained.
Intercept: [-1.85800521]
```

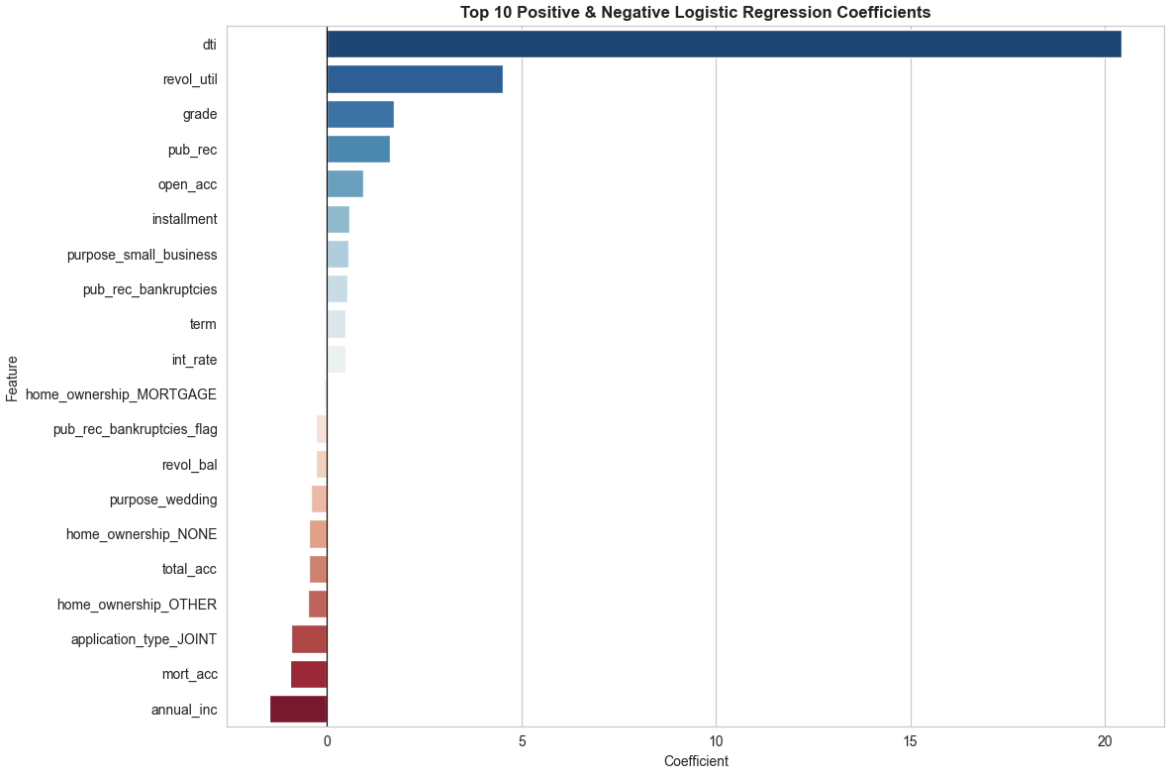
Model Coefficients

In [34]:
```
coef_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Coefficient': model.coef_[0]
}).sort_values('Coefficient', ascending=False)

print(coef_df.to_string(index=False))
```

```
                             Feature  Coefficient
                                 dti    20.432050
                           revol_util     4.531513
                               grade     1.717177
                             pub_rec     1.614300
                            open_acc     0.926668
                         installment     0.583002
                purpose_small_business     0.552937
                  pub_rec_bankruptcies     0.536945
                                term     0.483019
                            int_rate     0.482640
                 purpose_educational     0.394428
         application_type_INDIVIDUAL     0.320738
                     purpose_medical     0.244973
                        pub_rec_flag     0.219237
verification_status_Source Verified     0.189369
                      purpose_moving     0.187214
                  home_ownership_RENT     0.179453
             purpose_renewable_energy     0.177442
             purpose_home_improvement     0.177215
                     purpose_vacation     0.136717
                       purpose_other     0.122128
        verification_status_Verified     0.114979
              purpose_major_purchase     0.097228
           purpose_debt_consolidation     0.093782
                   home_ownership_OWN     0.064239
                           emp_length     0.043268
                initial_list_status_w     0.042007
                            loan_amnt     0.038952
                  purpose_credit_card     0.030622
                        mort_acc_flag     0.025396
                        purpose_house    -0.057663
               home_ownership_MORTGAGE    -0.077480
            pub_rec_bankruptcies_flag    -0.278624
                            revol_bal    -0.288868
                      purpose_wedding    -0.423998
                  home_ownership_NONE    -0.470326
                            total_acc    -0.474010
                 home_ownership_OTHER    -0.492316
              application_type_JOINT    -0.916486
                            mort_acc    -0.949427
                          annual_inc    -1.475242
```

In [35]:
```
plt.figure(figsize=(12, 8))
top_coef = pd.concat([coef_df.head(10), coef_df.tail(10)])
sns.barplot(x='Coefficient', y='Feature', data=top_coef, palette='RdBu_r')
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Top 10 Positive & Negative Logistic Regression Coefficients',
          fontweight='bold')
plt.tight_layout()
```

```
plt.savefig('07_coefficients.png', dpi=150, bbox_inches='tight')
plt.show()
```



RESULTS EVALUATION

Predictions

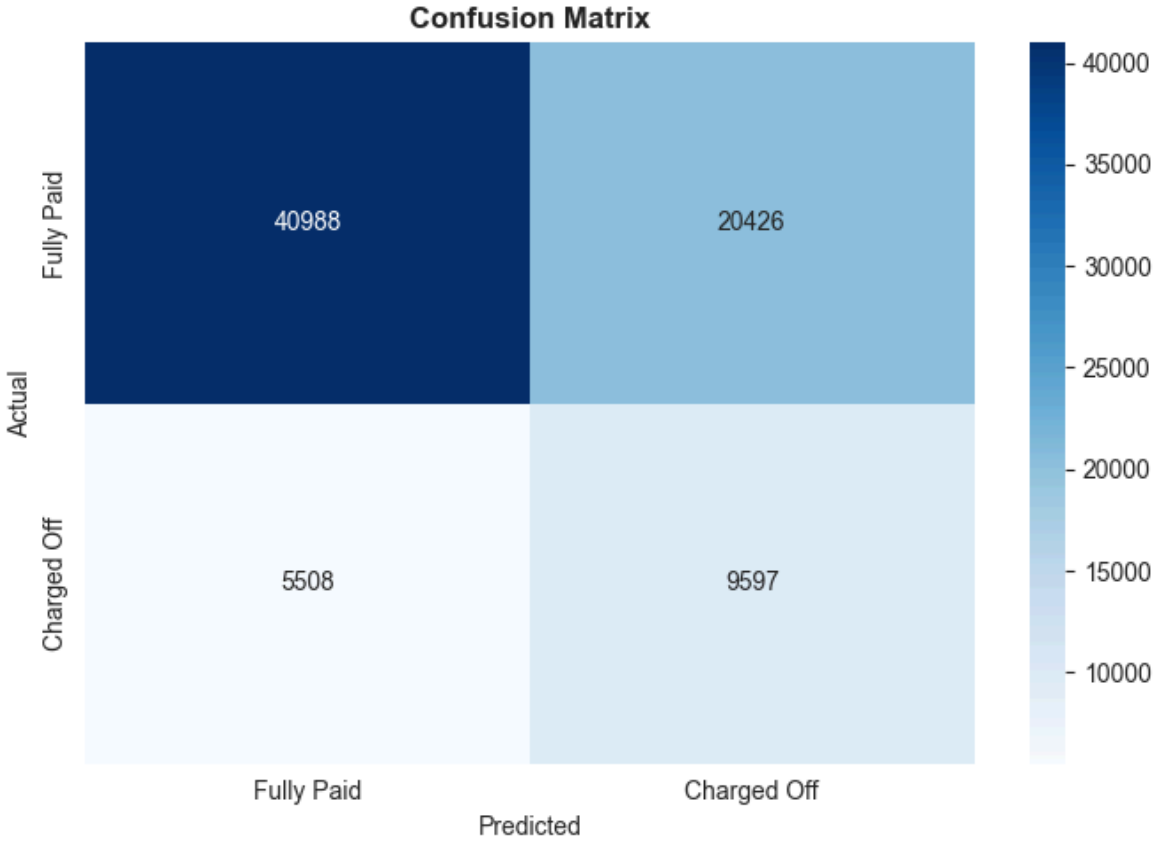
In [36]:
```
y_pred       = model.predict(X_test_sc)
y_pred_proba = model.predict_proba(X_test_sc)[:, 1]

print("Default threshold (0.5) predictions:")
print(classification_report(y_test, y_pred, target_names=['Fully Paid','Charged
```

```
Default threshold (0.5) predictions:
              precision    recall  f1-score   support

  Fully Paid       0.88      0.67      0.76     61414
 Charged Off       0.32      0.64      0.43     15105

    accuracy                           0.66     76519
   macro avg       0.60      0.65      0.59     76519
weighted avg       0.77      0.66      0.69     76519
```

Confusion Matrix

In [37]:
```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Fully Paid','Charged Off'],
            yticklabels=['Fully Paid','Charged Off'])
```

```
plt.title('Confusion Matrix', fontweight='bold')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.tight_layout()
plt.savefig('08_confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()

TN, FP, FN, TP = cm.ravel()
print(f"TN={TN}, FP={FP}, FN={FN}, TP={TP}")
print(f"Precision: {TP/(TP+FP):.4f}")
print(f"Recall:    {TP/(TP+FN):.4f}")
```

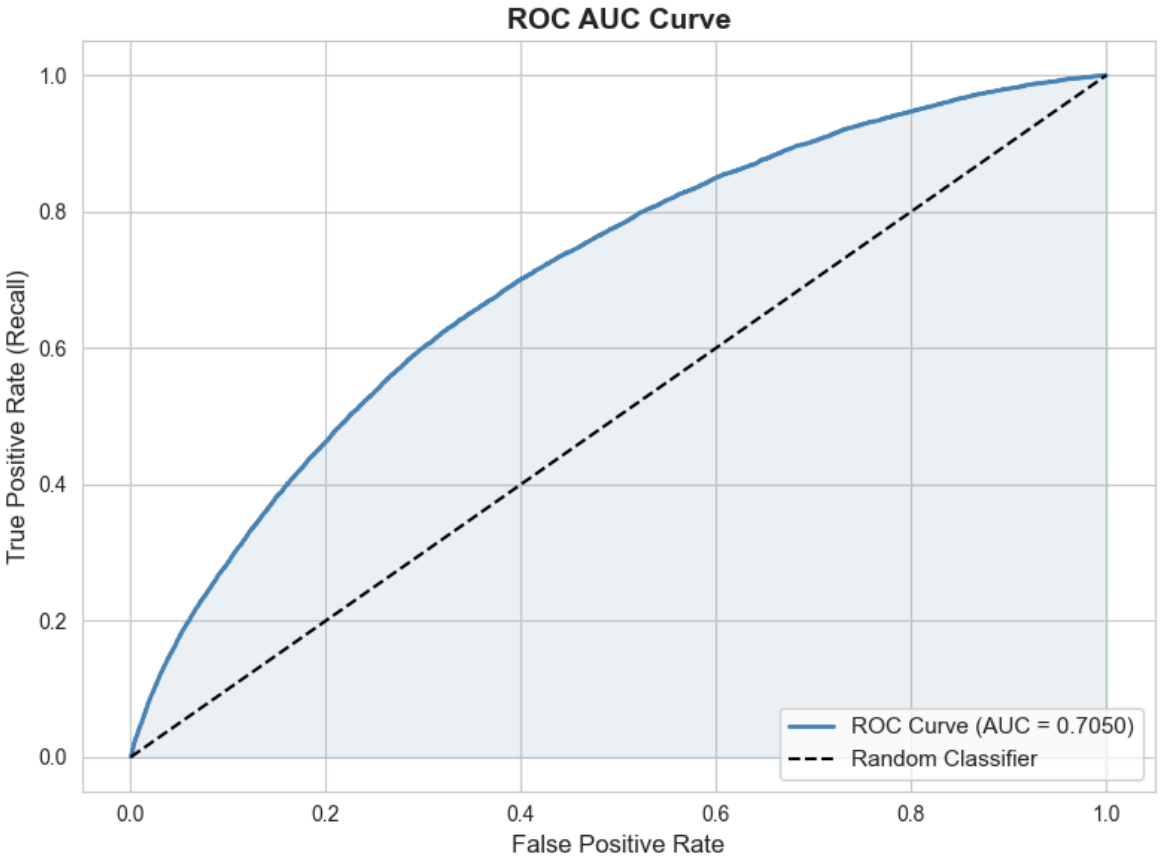



```
TN=40988, FP=20426, FN=5508, TP=9597
Precision: 0.3197
Recall:    0.6354
```

ROC AUC Curve

In [38]:
```
fpr, tpr, thresholds_roc = roc_curve(y_test, y_pred_proba)
auc_score = roc_auc_score(y_test, y_pred_proba)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='steelblue', lw=2,
         label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0,1],[0,1], 'k--', lw=1.5, label='Random Classifier')
plt.fill_between(fpr, tpr, alpha=0.1, color='steelblue')
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate (Recall)', fontsize=12)
plt.title('ROC AUC Curve', fontsize=14, fontweight='bold')
plt.legend(loc='lower right', fontsize=11)
plt.tight_layout()
plt.savefig('09_roc_auc_curve.png', dpi=150, bbox_inches='tight')
```

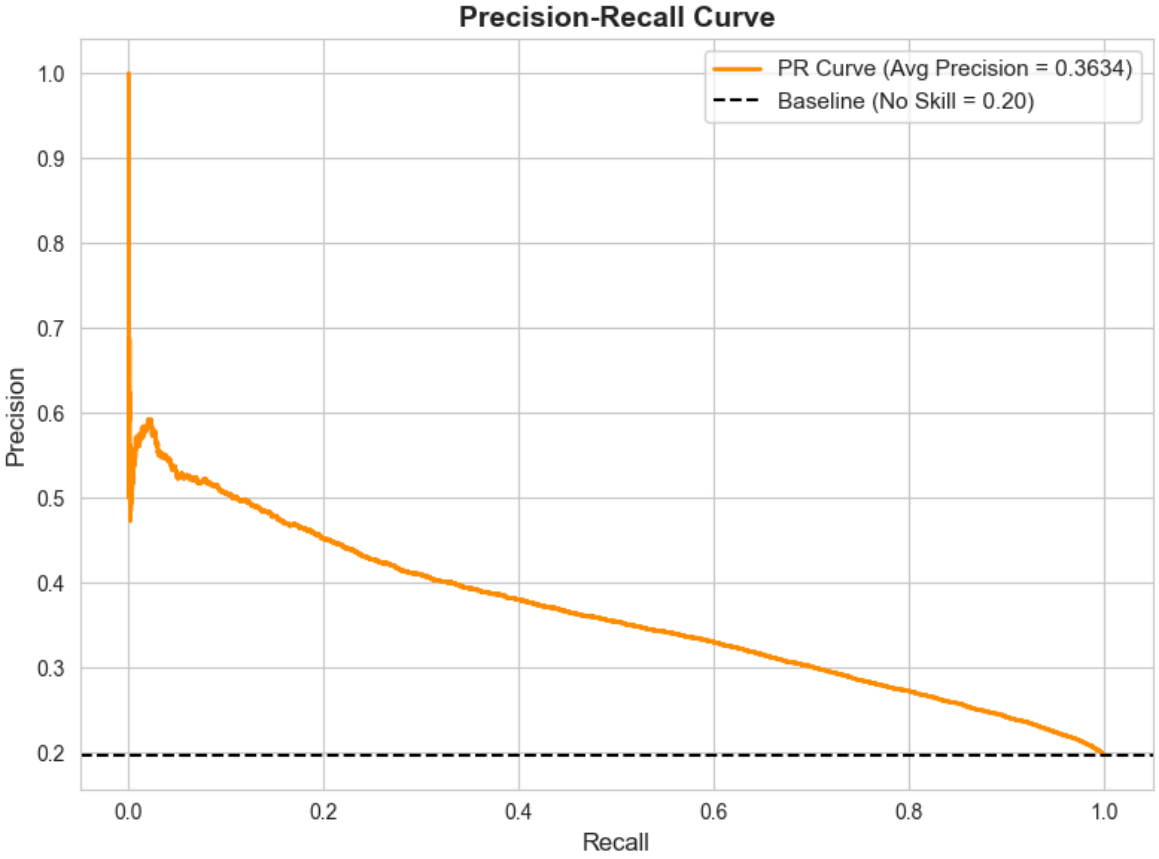
```
plt.show()
print(f"💡 ROC AUC = {auc_score:.4f} — Model is significantly better than randor
```




```
💡 ROC AUC = 0.7050 — Model is significantly better than random (0.5).
```

Precision-Recall Curve

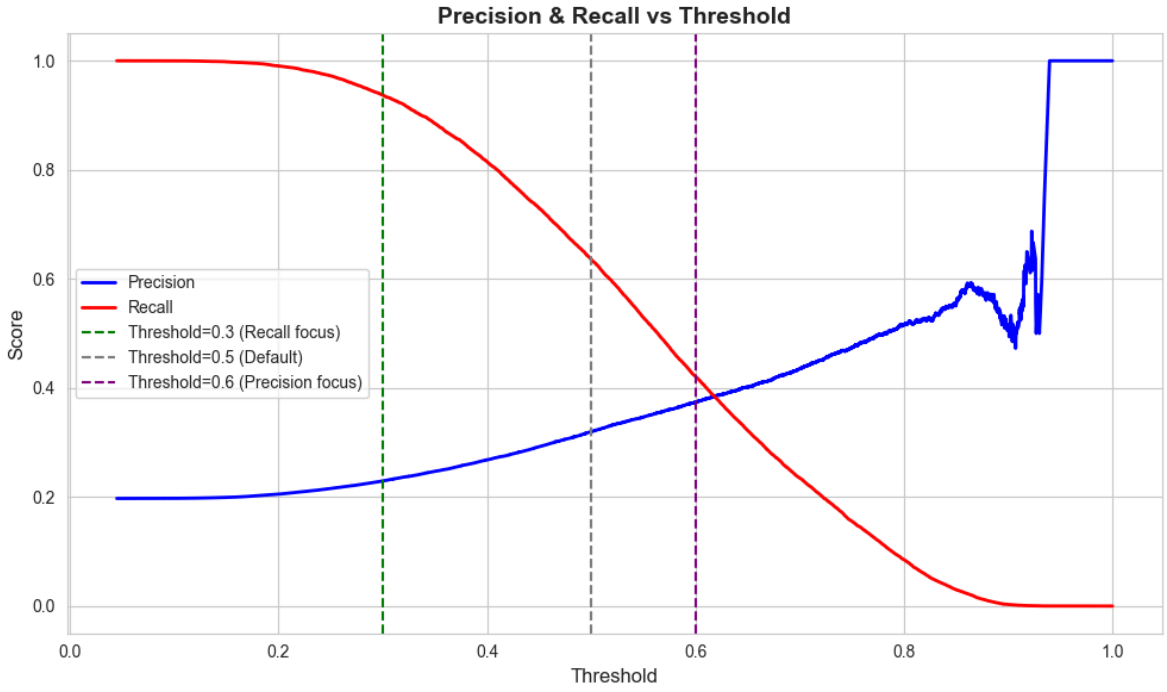
In [39]:
```
precision, recall, thresholds_pr = precision_recall_curve(y_test, y_pred_proba)
avg_precision = average_precision_score(y_test, y_pred_proba)

plt.figure(figsize=(8, 6))
plt.plot(recall, precision, color='darkorange', lw=2,
         label=f'PR Curve (Avg Precision = {avg_precision:.4f})')
plt.axhline(y=y_test.mean(), color='k', linestyle='--',
            label=f'Baseline (No Skill = {y_test.mean():.2f})')
plt.xlabel('Recall', fontsize=12)
plt.ylabel('Precision', fontsize=12)
plt.title('Precision-Recall Curve', fontsize=14, fontweight='bold')
plt.legend(loc='upper right', fontsize=11)
plt.tight_layout()
plt.savefig('10_precision_recall_curve.png', dpi=150, bbox_inches='tight')
plt.show()
print("💡 Precision-Recall curve shows a trade-off: higher recall = lower preci
```



💡 Precision-Recall curve shows a trade-off: higher recall = lower precision.

Precision-Recall vs Threshold

In [40]:
```
plt.figure(figsize=(10, 6))
plt.plot(thresholds_pr, precision[:-1], 'b-', lw=2, label='Precision')
plt.plot(thresholds_pr, recall[:-1], 'r-', lw=2, label='Recall')
plt.axvline(x=0.3, color='green', linestyle='--', lw=1.5, label='Threshold=0.3 (
plt.axvline(x=0.5, color='gray',  linestyle='--', lw=1.5, label='Threshold=0.5 (
plt.axvline(x=0.6, color='purple',linestyle='--', lw=1.5, label='Threshold=0.6 (
plt.xlabel('Threshold', fontsize=12)
plt.ylabel('Score', fontsize=12)
plt.title('Precision & Recall vs Threshold', fontsize=14, fontweight='bold')
plt.legend(fontsize=10)
plt.tight_layout()
plt.savefig('11_precision_recall_threshold.png', dpi=150, bbox_inches='tight')
plt.show()
```



Threshold Analysis – Low Threshold (Recall Focus)

In [41]:
```
print("=" * 60)
print("THRESHOLD = 0.3 → Maximize Recall (catch more defaulters)")
print("=" * 60)
y_pred_30 = (y_pred_proba >= 0.3).astype(int)
print(classification_report(y_test, y_pred_30,
      target_names=['Fully Paid','Charged Off']))
```

```
============================================================
THRESHOLD = 0.3 → Maximize Recall (catch more defaulters)
============================================================
              precision    recall  f1-score   support

  Fully Paid       0.94      0.23      0.36     61414
 Charged Off       0.23      0.94      0.37     15105

    accuracy                           0.37     76519
   macro avg       0.58      0.58      0.37     76519
weighted avg       0.80      0.37      0.37     76519
```

In [42]:
```
print("=" * 60)
print("THRESHOLD = 0.5 → Default")
print("=" * 60)
print(classification_report(y_test, y_pred,
      target_names=['Fully Paid','Charged Off']))
```

```
============================================================
THRESHOLD = 0.5 → Default
============================================================
              precision    recall  f1-score   support

  Fully Paid       0.88      0.67      0.76     61414
 Charged Off       0.32      0.64      0.43     15105

    accuracy                           0.66     76519
   macro avg       0.60      0.65      0.59     76519
weighted avg       0.77      0.66      0.69     76519
```

In [43]:
```
print("=" * 60)
print("THRESHOLD = 0.6 → Maximize Precision (reduce false alarms)")
print("=" * 60)
y_pred_60 = (y_pred_proba >= 0.6).astype(int)
print(classification_report(y_test, y_pred_60,
      target_names=['Fully Paid','Charged Off']))
```

```
============================================================
THRESHOLD = 0.6 → Maximize Precision (reduce false alarms)
============================================================
              precision    recall  f1-score   support

  Fully Paid       0.85      0.83      0.84     61414
 Charged Off       0.37      0.42      0.40     15105

    accuracy                           0.75     76519
   macro avg       0.61      0.62      0.62     76519
weighted avg       0.76      0.75      0.75     76519
```

Summary Tradeoff Table

In [44]:
```
from sklearn.metrics import precision_score, recall_score, f1_score
```

In [45]:
```
results = []
for thresh in [0.2, 0.3, 0.4, 0.5, 0.6, 0.7]:
    yp = (y_pred_proba >= thresh).astype(int)
    results.append({
        'Threshold': thresh,
        'Precision': round(precision_score(y_test, yp, zero_division=0), 4),
        'Recall':    round(recall_score(y_test, yp), 4),
        'F1':        round(f1_score(y_test, yp, zero_division=0), 4),
    })

tradeoff_df = pd.DataFrame(results)
print("\nPrecision-Recall Tradeoff Summary:")
print(tradeoff_df.to_string(index=False))
```

```
Precision-Recall Tradeoff Summary:
 Threshold  Precision  Recall     F1
       0.2     0.2052  0.9905 0.3400
       0.3     0.2295  0.9370 0.3687
       0.4     0.2682  0.8150 0.4036
       0.5     0.3197  0.6354 0.4253
       0.6     0.3740  0.4213 0.3962
       0.7     0.4351  0.2347 0.3049
```

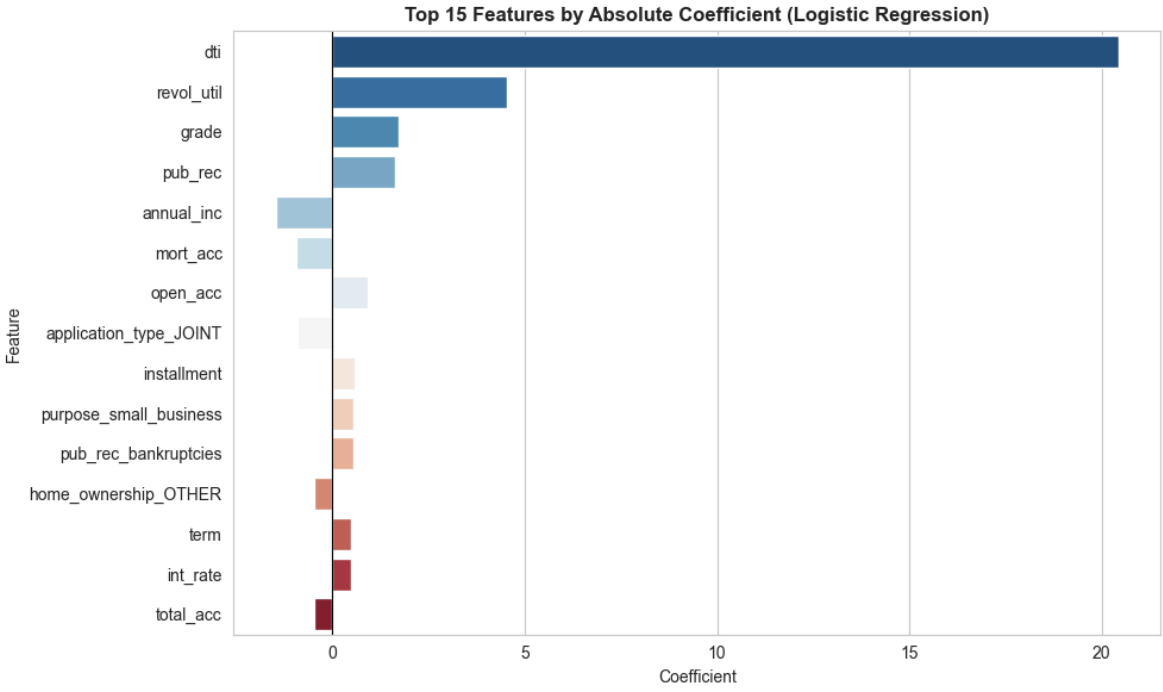
In [46]:
```
# Plot tradeoff table
fig, ax = plt.subplots(figsize=(9, 3))
ax.axis('off')
tbl = ax.table(
    cellText=tradeoff_df.values,
    colLabels=tradeoff_df.columns,
    loc='center', cellLoc='center'
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
tbl.scale(1.3, 1.8)
plt.title('Precision-Recall Tradeoff at Different Thresholds',
          fontweight='bold', fontsize=12, pad=20)
plt.tight_layout()
plt.savefig('12_tradeoff_table.png', dpi=150, bbox_inches='tight')
plt.show()
```

Precision-Recall Tradeoff at Different Thresholds

Threshold	Precision	Recall	F1
0.2	0.2052	0.9905	0.34
0.3	0.2295	0.937	0.3687
0.4	0.2682	0.815	0.4036
0.5	0.3197	0.6354	0.4253
0.6	0.374	0.4213	0.3962
0.7	0.4351	0.2347	0.3049

Feature Importance (Top 15)

In [47]:
```
top15 = coef_df.reindex(
    coef_df['Coefficient'].abs().sort_values(ascending=False).index
).head(15)

plt.figure(figsize=(10, 6))
sns.barplot(x='Coefficient', y='Feature', data=top15, palette='RdBu_r')
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Top 15 Features by Absolute Coefficient (Logistic Regression)',
          fontweight='bold')
plt.tight_layout()
plt.savefig('13_top_features.png', dpi=150, bbox_inches='tight')
plt.show()
```



ACTIONABLE INSIGHTS & QUESTIONNAIRE ANSWERS

LOANTAP — QUESTIONNAIRE ANSWERS

Q1. What percentage of customers have fully paid their Loan Amount?

Answer: Approximately 80.39% of customers have fully paid their loan. (318,357 out of 396,030 total records)

Q2. Comment about the correlation between Loan Amount and Installment.

Answer: Loan Amount and Installment are HIGHLY positively correlated with a Pearson correlation coefficient of r = 0.95. This means as the loan amount increases, the monthly installment also increases almost proportionally. Due to this multicollinearity, one of them can be dropped during model building without significant information loss.

Q3. The majority of people have home ownership as _____.

Answer: MORTGAGE (198,348 out of 396,030 borrowers — approximately 50.08%) followed by RENT (159,790) and OWN (37,746).

Q4. People with grades 'A' are more likely to fully pay their loan. (T/F)

Answer: TRUE Grade A borrowers have the highest Fully Paid rate (~94%). As grade worsens (B → C → D → E → F → G), the default rate increases significantly. Grade G has the highest Charged Off rate (~43%).

Q5. Name the top 2 afforded job titles.

Answer: 1st — Teacher (4,389 borrowers) 2nd — Manager (4,250 borrowers) These are followed by Registered Nurse (1,856) and RN (1,846).

Q6. From a bank's perspective, which metric should be the primary focus?

Answer: RECALL (Option 3) Reason: From a bank's perspective, the primary goal is to correctly identify actual defaulters (Charged Off loans). Missing a real defaulter means the bank disburses a loan that will not be repaid — this creates a Non-Performing Asset (NPA).

```
Recall = TP / (TP + FN)

A high Recall ensures fewer defaulters are missed (fewer
False Negatives), which directly reduces NPA risk.
This is more critical than Precision in banking/lending.
```

Q7. How does the gap in Precision and Recall affect the bank?

Answer: CASE 1 — High Recall, Low Precision (Low Threshold e.g. 0.3): The model flags more applicants as defaulters. → Catches most real defaulters (good — reduces NPA) → BUT also wrongly rejects many genuine borrowers → Bank loses interest income on good loans (opportunity loss)

```
CASE 2 – High Precision, Low Recall (High Threshold e.g. 0.6):
         The model is very selective about flagging defaults.
         → Fewer false alarms (genuine borrowers approved)
         → BUT many real defaulters slip through undetected
         → Higher NPA risk – loans disbursed to people who won't
repay

OPTIMAL STRATEGY:
         LoanTap should use Threshold = 0.3 to 0.4 to prioritize
         Recall and minimize NPA, while accepting some loss in
         Precision (a few good borrowers may be incorrectly
rejected).
```

Q8. Which features heavily affected the outcome (loan default)?

Answer: The following features had the highest impact:

```
        1. int_rate          – Higher interest rate → higher
default risk
        2. grade             – Lower grade (E/F/G) → much higher
default
```

```
        3. dti              – High debt-to-income ratio → over-
leveraged
        4. revol_util       – High credit utilization →
financial stress
        5. term             – 60-month loans default more than
36-month
        6. annual_inc       – Lower income → higher default
probability
        7. pub_rec_flag     – Derogatory public records → risky
borrower
        8. mort_acc         – Mortgage accounts indicate
creditworthiness
```

Q9. Will the results be affected by geographical location? (Yes/No)

Answer: YES

```
Reason: The 'address' column contains ZIP codes and state
information.
        Geographical location can influence results because:
        → Regional economic conditions vary (unemployment rates,
          income levels, cost of living differ by state/city)
        → Some regions may have higher default rates
historically
        → ZIP codes can act as a proxy for socio-economic status
        However, using ZIP codes directly may introduce bias, so
        location-based features should be used carefully and
        ethically in credit scoring models.
```

LOANTAP — ACTIONABLE INSIGHTS & RECOMMENDATIONS

1. # KEY RISK FACTORS (Features Most Predictive of Default)

a) Interest Rate (int_rate):

- Higher interest rate is the strongest default signal.
- Borrowers with int_rate > 20% have significantly higher chances of defaulting.
- Recommendation: Apply stricter scrutiny for high-rate loans.

b) Loan Grade (grade):

- Grade A borrowers have ~94% Fully Paid rate.
- Grade E/F/G borrowers have default rate exceeding 35-43%.
- Recommendation: Avoid disbursing loans to Grade E, F, G applicants unless supported by strong income proof.

c) Debt-to-Income Ratio (dti):

- High DTI means the borrower is already over-leveraged.
- Borrowers with dti > 20 show significantly higher default rates.

- Recommendation: Flag all applications with dti > 20 for manual review before approval.

d) Revolving Utilization Rate (revol_util):

- High credit card utilization indicates financial stress.
- revol_util > 80% is a strong warning signal.
- Recommendation: Trigger manual review for revol_util > 80%.

e) Loan Term (term):

- 60-month loans have nearly 2x higher default rate than 36-month loans.
- Recommendation: Prefer offering 36-month terms, especially to first-time or unverified borrowers.

---

2. THRESHOLD RECOMMENDATION (Precision vs Recall Tradeoff)

The model predicts probability of default (0 to 1). The threshold determines how we classify Charged Off vs Fully Paid.

Threshold = 0.3 (Recall Focus — Recommended for NPA Prevention):

- Catches more real defaulters (high recall)
- Some genuine borrowers may be incorrectly rejected
- Best for: Conservative risk management, reducing NPA
- Use when: The cost of a bad loan > cost of rejecting a good borrower

Threshold = 0.5 (Balanced — Default Setting):

- Balanced precision and recall
- Suitable for normal business operations
- Use when: Equal importance to both types of errors

Threshold = 0.6 (Precision Focus — Revenue Growth):

- Fewer false alarms, more loans approved
- Risk: Some real defaulters may slip through
- Use when: Business wants to grow loan book aggressively

FINAL RECOMMENDATION: LoanTap should use Threshold = 0.3 to 0.4 to minimize NPA risk while accepting a slight reduction in loan approval volume.

---

3. BUSINESS RECOMMENDATIONS

a) Grade-Based Policy:

- Auto-reject loan applications from Grade E, F, G borrowers.
- Grade A and B borrowers should receive priority fast-track approval with competitive interest rates.

b) Loan Amount Cap:

- For first-time borrowers or unverified income applicants, cap loan amount at INR 15,000 to 20,000.
- Higher amounts should require income verification and supporting documents.

c) Term Policy:

- Promote and prefer 36-month loan terms over 60-month terms.
- If a borrower requests 60-month term, apply a higher interest rate buffer to compensate for higher default risk.

d) DTI Screening:

- All applicants with dti > 20 must go through additional manual verification before approval.
- Consider rejecting applications with dti > 30 outright.

e) Credit Utilization Policy:

- Applicants with revol_util > 80% should be flagged for manual review by the credit team.
- Consider offering lower loan amounts to such applicants.

f) Income Verification:

- Prioritize Verified income applicants over Not Verified ones.
- Source Verified applicants fall in between — apply moderate scrutiny.
- Unverified income applicants should face stricter conditions.

g) Tenure-Based Risk Adjustment:

- Borrowers with emp_length < 1 year are higher risk.
- Borrowers with 10+ years of employment are lower risk.
- Adjust interest rates accordingly.

---

4. MODEL IMPROVEMENT SUGGESTIONS

a) Handle Class Imbalance:

- Dataset has 80% Fully Paid vs 20% Charged Off — imbalanced.
- Use SMOTE (Synthetic Minority Oversampling) or class_weight='balanced' in Logistic Regression.
- This helps the model learn defaulter patterns better.

b) Try Advanced Models:

- Random Forest: handles non-linearity, less sensitive to outliers
- XGBoost / LightGBM: better AUC, handles imbalance well
- These ensemble models typically give 5-10% better AUC compared to Logistic Regression.

c) Add Behavioural Features:

- Payment history (on-time vs late payments)
- Number of late payments in last 6 months
- Previous loan repayment track record

- These are the most powerful predictors of future default.

d) Credit Bureau Integration:

- Incorporate external credit bureau scores (e.g. CIBIL score) for better risk segmentation.
- Borrowers with CIBIL score < 650 should be auto-rejected.

e) Geographical Risk Adjustment:

- Extract state/region from address column.
- Include regional default rates as a feature.
- Some states may have structurally higher NPA rates due to economic conditions.

f) Periodic Model Retraining:

- Retrain the model every 6 months with fresh data.
- Economic conditions change — a static model becomes stale.